

Crear una prueba nueva con el generador

Los dos enfoques (receta y asistente), paso a paso y con ejemplos completos — hasta ejecutar las pruebas.

A

Escribes la receta (spec JSON)

o respondes el asistente

1

Ejecutas el generador

crea page, steps, feature y datos

2

Completas / verificas selectores

donde diga // TODO

3

Corres las pruebas

npm run test:qa

¿Qué hace el generador y por qué usarlo?

Crear una prueba nueva a mano implica escribir 4 o 5 archivos coordinados entre sí (la página, los pasos, el feature, los datos). Es repetitivo y fácil de equivocarse. El generador hace TODO ese esqueleto por ti a partir de los insumos del caso de prueba, y deja archivos que YA corren.

Qué archivos crea por ti

Archivo	Para qué sirve
src/pages/<Modulo>Page.ts	La "página": locators y métodos de acción ya cableados.
src/test/stepsDefinitions/<modulo>/...	Los pasos (Given/When/Then) conectados a la página.
src/test/features/<modulo>/<Modulo> feature	El escenario en lenguaje Gherkin.
jsonData/<env>/<modulo>.json	Los datos de prueba.
core/interfaces/<Modulo>Data.ts	El "tipo" de los datos (si hay datos).



INFO — No hay que registrar nada

Cucumber descubre features, pasos y datos por convención. Apenas el generador crea los archivos, ya forman parte de la suite.



LISTO — La regla de oro: sin magia frágil

Las acciones se conectan a métodos que el framework YA conoce (un catálogo cerrado). Si pides algo que no está en el catálogo, el generador falla con un mensaje claro ANTES de escribir nada. Nunca "adivina".

Los dos caminos

ENFOQUE A

La receta (spec JSON)

Escribes un archivo .spec.json con los insumos del caso y lo pasas al generador. Versionable y revisable en equipo.

```
npm run new:module -- specs/x.spec.json
```

ENFOQUE B

El asistente (preguntas)

Ejecutas el comando sin nada y respondes preguntas. Al final guarda el mismo spec y genera todo.

```
npm run new:module
```

Antes de empezar (1 minuto)

El proyecto ya debe estar instalado (dependencias y navegador). Si es una máquina nueva, primero haz esto una sola vez:

```
# Estar dentro de la carpeta del proyecto
PS> cd "C:\...\Playwright_Web_Automation"
# Instalar dependencias y navegador (solo la 1a vez)
PS> npm install
PS> npx playwright install chromium
```

Preparación inicial

**CONSEJO — Verifica que estés bien parado**

Si al escribir "dir" (Windows) ves el archivo package.json, estás en la carpeta correcta y puedes continuar.

Anatomía del spec (la receta)

El spec es un archivo JSON. Tiene unos pocos campos obligatorios y varios opcionales. Esta es la referencia completa; en el Enfoque A lo armamos juntos.

Campos obligatorios

Campo	Tipo	Qué es
module	texto	Nombre del módulo. Se convierte a PascalCase y define todos los nombres.
route	texto	Ruta tras la URL base. Debe empezar con "/"
scenarios	lista	Al menos 1 escenario (ver más abajo).

Campos opcionales

Campo	Default	Qué es
description	= module	Texto que va en la línea "Feature:"
envs	["qa"]	Para qué ambientes crear el archivo de datos.
locators	[]	Los elementos de la pantalla (campos, botones...).
anchorLocator	1er locator	El elemento que confirma que la página cargó.
actions	[]	Acciones reutilizables -> se vuelven métodos de la página.
data	[]	Filas de datos de prueba (cada una con "id" único).

Cómo se escribe un locator

Cada locator es la "dirección" de un elemento. Tiene name (cómo lo llamas), by (estrategia) y value:

by	Se usa para	Ejemplo de value
placeholder	Campo por su texto de ayuda	"Search"
role	Botón/elemento por su rol (admite "name_value")	"button" + name_value "Add"
label	Campo por su etiqueta	"Campaign Name"
text	Elemento por su texto visible	"Guardar"
testid	Atributo data-testid	"submit-btn"
css	Selector CSS directo	".oxd-toast"
xpath	Expresión XPath (page.locator xpath=)	"//button[@type=submit]"

El catálogo de acciones (cerrado)

Una "action" se convierte en un método de la página. Solo puedes usar estos tipos (por eso decimos catálogo "cerrado"): cada uno se conecta a algo que el framework ya sabe hacer. La columna "params" indica cuántos valores entre comillas debe tener el paso que lo use.

type	Qué hace	Campos extra	params
goto	Navega a una ruta (ej. login)	path (+ anchor opc.)	0
fill	Escribe en un campo	locator	1
click	Hace clic	locator	0
select	Elige opción de un <select>	locator	1
check	Marca/desmarca checkbox	locator	0
choose	Selecciona un registro/fila	locator	0
upload	Sube un archivo	locator	1
assertVisible	Verifica que un elemento se ve	locator	0
assertText	Verifica texto en un elemento	locator + expected	0
assertAllText	Verifica que varios textos sean iguales	locator + expected	0
assertUrlContains	Verifica que la URL contiene algo	locator + fragment	0
assertUrlMatches	Verifica que la URL coincide con un patrón	pattern	0

Escenarios y pasos (steps)

- Cada escenario tiene: title, tags (opcional), jira (opcional) y steps.
- Cada step tiene: kw (Given / When / And / But / Then) y text (la frase).
- En el text, usa "comillas dobles" para los valores variables: se convierten en {string}.
- bind (opcional): "navigate" llama a navegar; el nombre de una action conecta ese método; sin bind genera un stub pendiente.
- dataId (opcional): el valor entre comillas se trata como id de "data", y se pasa el campo indicado al método. Ej: dataId "name".



ATENCIÓN — Evita estos caracteres en el text

No uses () { } / \ en el texto del paso. Usa comillas solo para los valores que cambian. Si no, el generador te avisará.

Background: precondition común (ej. login)

Si todos los escenarios necesitan lo mismo antes (típico: iniciar sesión), ponlo en "background". Se ejecuta antes de cada escenario del módulo.

Para no duplicar el login: defínelo UNA vez (módulo Login) y en los demás módulos repite las MISMAS frases en el background, SIN bind. El generador detecta que ya existen y las reutiliza (no las regenera), evitando el error "Multiple step definitions match".

```
background (reutiliza el login de otro módulo)
```

```
"background": [  
  { "kw": "Given", "text": "el usuario está en la página de login" },
```

```
{ "kw": "And",    "text": "ingresa el usuario del registro \"valido\"" },
{ "kw": "And",    "text": "ingresa la contraseña del registro \"valido\"" },
{ "kw": "And",    "text": "hace clic en iniciar sesión" }
]
```



INFO — Usa frases únicas por módulo

Para los pasos PROPIOS de un módulo, incluye el nombre del módulo en la frase (ej. "se muestra el título del módulo PIM"). Cucumber comparte los steps globalmente; una frase repetida con sentido distinto causaría conflicto.

Cómo se conecta todo: evento, elemento y valor

Esta es la duda más común: ¿qué evento corre, sobre qué elemento, y de dónde sale el valor (el que se escribe o el que se valida)? Nada se decide solo: todo está escrito en el spec y el generador solo lo conecta.

Una acción tiene 3 piezas

- type -> QUÉ evento/método se ejecuta (del catálogo). Ej: fill = escribir, click = clic, assertText = verificar texto.
- locator -> SOBRE QUÉ elemento actúa. Apunta por nombre a un locator que definiste.
- el valor -> depende del tipo de acción (ver la tabla).

¿De dónde sale el valor?

Tipo de acción	¿Valor?	¿De dónde sale?
fill, select, upload	Sí (1)	En tiempo de ejecución, desde el STEP: el texto entre comillas. Si el step tiene dataId, ese texto es un id y se pasa data.<campo>.
click, check, choose	No	— (no recibe valor)
assertText, assertAllText	Sí	FIJO en el spec: el campo "expected".
assertUrlContains	Sí	FIJO en el spec: el campo "fragment".
assertUrlMatches	Sí	FIJO en el spec: el campo "pattern".



INFO — Por eso el generador cuenta las comillas

El catálogo dice fill = 1 valor y assertText = 0. Así, un paso de fill DEBE traer una comilla (el valor a escribir) y uno de assertText, cero (porque el esperado ya está en el spec). Si no coincide, falla antes de generar.

Traza 1 — entrada (un fill que usa datos)

Feature

And ingresa el nombre de la campaña "happy-001" (bind: fillName, dataId: name)



Step

Captura arg0 = "happy-001" (el valor entre comillas)



dataId

Busca en campanias.json la fila con id = "happy-001"



Valor

Toma el campo "name" -> "Campaña Black Friday"



Página

page.fillName("Campaña Black Friday")



Base

fillField(this.campaignName, "Campaña Black Friday")



Navegador

Escribe ese texto en el elemento campaignName (label "Campaign Name")



CONSEJO — Sin dataId, el valor es literal

Si el paso no tiene dataId, se pasa el texto entre comillas tal cual, sin buscar en los datos.

Traza 2 — aserción (el valor esperado viene del spec)

Spec

assertSaved -> type: assertText, locator: successToast, expected: "Successfully Saved"



Feature

Then se muestra el mensaje de campaña guardada (0 comillas)



Página

page.assertSaved()



Base

assertLocatorText(this.successToast, "Successfully Saved") <- el texto vino del spec



Navegador

Verifica que el elemento successToast CONTIENE "Successfully Saved"



LISTO — En una frase

El type elige el evento, el locator elige el elemento, y el valor viene del STEP (acciones de entrada) o del SPEC (aserciones).

A

Enfoque A — Escribir la receta (spec)

Vamos a crear, de principio a fin, una prueba para un módulo "Campanias". No dejamos nada afuera.

- 1 Crea la carpeta specs/ (si no existe) y un archivo nuevo. Lo más fácil: copia el ejemplo.

Crear el spec

```
# Copiar la plantilla de ejemplo a tu nuevo spec (Windows)
PS> copy specs\example.spec.json specs\campanias.spec.json
# En Mac: cp specs/example.spec.json specs/campanias.spec.json
```

- 2 Abre specs/campanias.spec.json con el Bloc de notas / VS Code y pon el módulo y la ruta:

specs/campanias.spec.json (parte 1)

```
{
  "module": "Campanias",
  "route": "/web/index.php/campaign/list",
  "description": "Gestión de campañas del contact center",
  "envs": ["qa"]
}
```

- 3 Agrega los locators (los elementos de la pantalla) y cuál es el "ancla" que confirma que cargó:

locators

```
"locators": [
  { "name": "searchInput", "by": "placeholder", "value": "Search" },
  { "name": "addButton", "by": "role", "value": "button", "name_value": "Add" },
  { "name": "campaignName", "by": "label", "value": "Campaign Name" },
  { "name": "successToast", "by": "css", "value": ".oxd-toast" }
],
"anchorLocator": "addButton",
```

- 4 Agrega las acciones (se vuelven métodos de la página). Cada una usa un locator del paso anterior:

actions

```
"actions": [
  { "name": "search", "type": "fill", "locator": "searchInput", "label": "campo de búsqueda" },
  { "name": "clickAdd", "type": "click", "locator": "addButton", "label": "botón Add" },
  { "name": "fillName", "type": "fill", "locator": "campaignName", "label": "nombre de campaña" },
  { "name": "assertSaved", "type": "assertText", "locator": "successToast",
    "expected": "Successfully Saved", "label": "verifica toast de guardado" }
],
```

- 5 Agrega los datos de prueba. Cada fila necesita un "id" único:

data

```
"data": [
  { "id": "happy-001", "name": "Campaña Black Friday" },
  { "id": "happy-002", "name": "Campaña Cyber Monday" }
],
```

- 6 Finalmente, los escenarios. Conecta cada paso con bind (y dataId cuando uses datos):

scenarios (cierra el archivo)

```
"scenarios": [
  {
    "title": "Crear una campaña con datos válidos",
```

```

    "tags": ["Regresion"], "jira": "",
    "steps": [
      { "kw": "Given", "text": "el usuario está en la página de campañas",
        "bind": "navigate" },
      { "kw": "When", "text": "hace clic en agregar campaña",
        "bind": "clickAdd" },
      { "kw": "And", "text": "ingresa el nombre de la campaña \"happy-001\"",
        "bind": "fillName", "dataId": "name" },
      { "kw": "Then", "text": "se muestra el mensaje de campaña guardada",
        "bind": "assertSaved" }
    ]
  },
  {
    "title": "Buscar una campaña existente",
    "tags": ["Regresion"], "jira": "",
    "steps": [
      { "kw": "Given", "text": "el usuario está en la página de campañas",
        "bind": "navigate" },
      { "kw": "When", "text": "busca la campaña \"happy-002\"",
        "bind": "search", "dataId": "name" },
      { "kw": "Then", "text": "se listan los resultados de la búsqueda" }
    ]
  }
]
}

```



INFO — El último paso no tiene bind — y está bien

El paso "se listan los resultados" no tiene bind a propósito: el generador creará un stub pendiente (return 'pending') que tú completas después. Así no obtienes un falso verde.

Ejecuta el generador

Generar (Enfoque A)

```

# Generar todos los archivos a partir del spec
PS> npm run new:module -- specs/campanias.spec.json

```

Verás esta salida. La frase clave es "dry-run OK": significa que los archivos compilan y no hay pasos sin definir.

salida del generador

```

Archivos generados:
  src/pages/CampaniasPage.ts
  src/test/stepsDefinitions/campanias/CampaniasStepDefinitions.ts
  src/test/features/campanias/Campanias.feature
  jsonData/qa/campanias.json
  core/interfaces/CampaniasData.ts

Validando (prettier + cucumber --dry-run)...
  dry-run OK: no hay steps sin definir y los archivos compilan.

Siguiente paso: abre CampaniasPage.ts y completa los // TODO.

```

Completa lo que quedó pendiente

El paso sin bind generó esto en el archivo de steps. Reemplaza el cuerpo por una verificación real (por ejemplo, agregando una action "assertText" sobre la tabla de resultados y volviendo a generar con --force, o escribiendo el método a mano):

CampaniasStepDefinitions.ts (lo pendiente)

```
Then('se listan los resultados de la búsqueda', async function (this: CustomWorld) {  
  // TODO: implementar este paso. "se listan los resultados de la búsqueda"  
  return 'pending';  
});
```



CONSEJO — Y siempre: verifica los selectores

El generador pone los locators que escribiste, pero debes confirmar que coinciden con tu app real (abre la pantalla y revisa placeholder, label, etc.). Eso es lo único "humano" que queda.

Corre las pruebas (Enfoque A listo)

Ejecutar

```
# Ejecuta TODAS las pruebas (incluye tu módulo nuevo)  
PS> npm run test:qa  
# 0 solo tu feature recién creado:  
PS> npm test -- src/test/features/campanias/Campanias.feature
```

B

Enfoque B — El asistente (preguntas)

Si prefieres no escribir el JSON, ejecuta el comando SIN argumentos y el asistente te hará preguntas. Al final guarda el mismo spec y genera todo. Vamos a crear el MISMO módulo "Campanias" respondiendo preguntas.

Iniciar el asistente

```
PS> npm run new:module
```

A continuación, la conversación completa. En azul la pregunta, en verde lo que TÚ escribes. Una respuesta vacía (solo Enter) termina la lista actual.

asistente – datos y locators

```
-- Generador de módulo (wizard) --
Enter vacío deja el campo opcional vacío.

Nombre del módulo (PascalCase): Campanias
Route (path tras BASE_URL): /web/index.php/campaign/list
Descripción (opcional): Gestión de campañas
Envs separados por coma [qa]: qa

Locators. Enter en "name" para terminar.
  locator name (camelCase): searchInput
  by [role/placeholder/label/text/testid/css]: placeholder
  value: Search
  locator name (camelCase): addButton
  by: role
  value: button
  accessible name (opcional, para role): Add
  locator name (camelCase): campaignName
  by: label
  value: Campaign Name
  locator name (camelCase): successToast
  by: css
  value: .oxd-toast
  locator name (camelCase): (Enter para terminar)
anchorLocator para navigateTo [searchInput]: addButton
```

asistente – acciones

```
Acciones (fill, click, select, check, choose, upload,
assertText, assertAllText, assertUrlContains, assertUrlMatches).
  action name: search
  type: fill
  locator: searchInput
  label (opcional): campo de búsqueda
  action name: clickAdd
  type: click
  locator: addButton
  label (opcional): botón Add
  action name: fillName
  type: fill
  locator: campaignName
  label (opcional): nombre de campaña
```

```
action name: assertSaved
type: assertText
locator: successToast
expected (texto esperado): Successfully Saved
label (opcional): verifica toast
action name: (Enter para terminar)
```

asistente – escenarios

Datos. Campos como "campo=valor", coma-separados.

```
data id: happy-001
campos: name=Campaña Black Friday
data id: happy-002
campos: name=Campaña Cyber Monday
data id: (Enter para terminar)
```

Escenarios. Enter en "title" para terminar.

```
Scenario title: Crear una campaña con datos válidos
tags (coma-separados, sin @): Regresion
jira key (opcional): (Enter)
```

Steps. Enter en "text" para terminar el escenario.

```
kw [Given/When/And/But/Then]: Given
text: el usuario está en la página de campañas
bind: navigate
kw: When
text: hace clic en agregar campaña
bind: clickAdd
kw: And
text: ingresa el nombre de la campaña "happy-001"
bind: fillName
dataId (campo de data a pasar, opcional): name
kw: Then
text: se muestra el mensaje de campaña guardada
bind: assertSaved
kw: (Enter termina el escenario)
```

```
Scenario title: (Enter termina)
```

asistente – resultado

```
¿Guardar el spec en specs/campanias.spec.json? [S/n]: S
Spec guardado en specs/campanias.spec.json
```

```
Archivos generados: (los mismos 5 del Enfoque A)
dry-run OK: no hay steps sin definir y los archivos compilan.
```



LISTO — A partir de aquí, igual que el Enfoque A

El asistente guardó specs/campanias.spec.json y generó los mismos archivos. Verifica los selectores, completa cualquier // TODO y corre las pruebas con `npm run test:qa`.

Garantías y errores comunes

El generador está hecho para fallar PRONTO y CLARO, antes de tocar tus archivos. Estas son sus protecciones:

- Valida todo el spec antes de escribir. Si algo está mal, no crea ningún archivo.
- No sobrescribe archivos existentes salvo que agregues `--force`.
- Tras generar, formatea y corre cucumber `--dry-run` para probar que todo compila y no hay pasos sin definir.

Si ves un error, búscalo aquí

Mensaje / Causa	Cómo resolverlo
"module" inválido	Usa solo letras y números (ej. Campañas, no "campañas!").
route debe empezar con "/"	Pon la ruta completa tras la URL, ej. /web/index.php/campaign/list.
bind "X" no es una acción definida	El bind debe ser "navigate" o el name de una action que exista en "actions".
tiene N comillas pero la acción espera M	El nº de "valores entre comillas" del text debe igualar los params de la acción (ver catálogo).
dataId "X" no existe en TODAS las filas	El campo de dataId debe existir en cada fila de "data".
Estos archivos ya existen	Vuelve a correr con <code>--force</code> para sobrescribir, o cambia el nombre del módulo.

Regenerar con `--force`

```
# Sobreescribir archivos existentes a propósito
PS> npm run new:module -- specs/campanias.spec.json --force
```

Ejecutar las pruebas (resumen)

Con cualquiera de los dos enfoques, una vez generados los archivos, así corres las pruebas:

Comando	Qué hace
<code>npm run test:qa</code>	Corre TODAS las pruebas en el ambiente qa.
<code>npm run test:qa:video</code>	Igual, pero graba video de cada escenario.
<code>npm test -- --tags @Regresion</code>	Solo los escenarios marcados con @Regresion.
<code>npm test -- src/test/features/campanias/Campanias.feature</code>	Solo ese archivo de pruebas.

Checklist final

- 1 El spec está completo (module, route y al menos un escenario).
- 2 Ejecutaste el generador y viste "dry-run OK".
- 3 Verificaste que los locators coinciden con tu app real.
- 4 Completaste los pasos // TODO (si los hubo).

5 Corriste `npm run test:qa` y revisaste el reporte (verde = pasó).



LISTO — ¡Eso es todo!

Con este manual cualquiera del equipo puede crear una prueba nueva con el generador (por receta o por preguntas) y ejecutarla. Guárdalo junto a `docs/GENERATOR.md` y `specs/example.spec.json`.